University of Asia Pacific

Department of Computer Science and Engineering

CSE 437: Robotics Lab

LAB REPORT

Experiment Number: 04

Experiment Title: Design and Implementation of a Self-Balancing Robot

Submitted by:

Team Name : BOD

Name : H.M. Tahsin Sheikh

Student ID : 22201243

Section : E1

Submitted to:

A S Zaforullah Momtaz
Assistant Professor,
Department of Computer Science and Engineering

Date of Submission: 07/03/2026

1. Experiment Name

Design and Implementation of a Self-Balancing Robot using Arduino and MPU6050 Sensor

2. Objective

The objective of this experiment is to design and implement a two-wheel self-balancing robot using a microcontroller and motion sensors. The robot should be able to maintain its upright position automatically by detecting tilt using an accelerometer and gyroscope sensor and correcting its balance through motor control using a PID algorithm.

3. Apparatus / Hardware & Software Requirements

Hardware Components:

- Arduino Uno (ATMEGA328P Microcontroller)
- MPU6050 (Accelerometer + Gyroscope Sensor)
- L298N Motor Driver Module
- Two DC Motors
- Robot Chassis with Wheels
- Breadboard
- Jumper Wires
- Lithium Battery / Power Supply
- Motor Mounts

Software:

- Arduino IDE
- TinkerCad

4. Circuit Diagram / Schematic

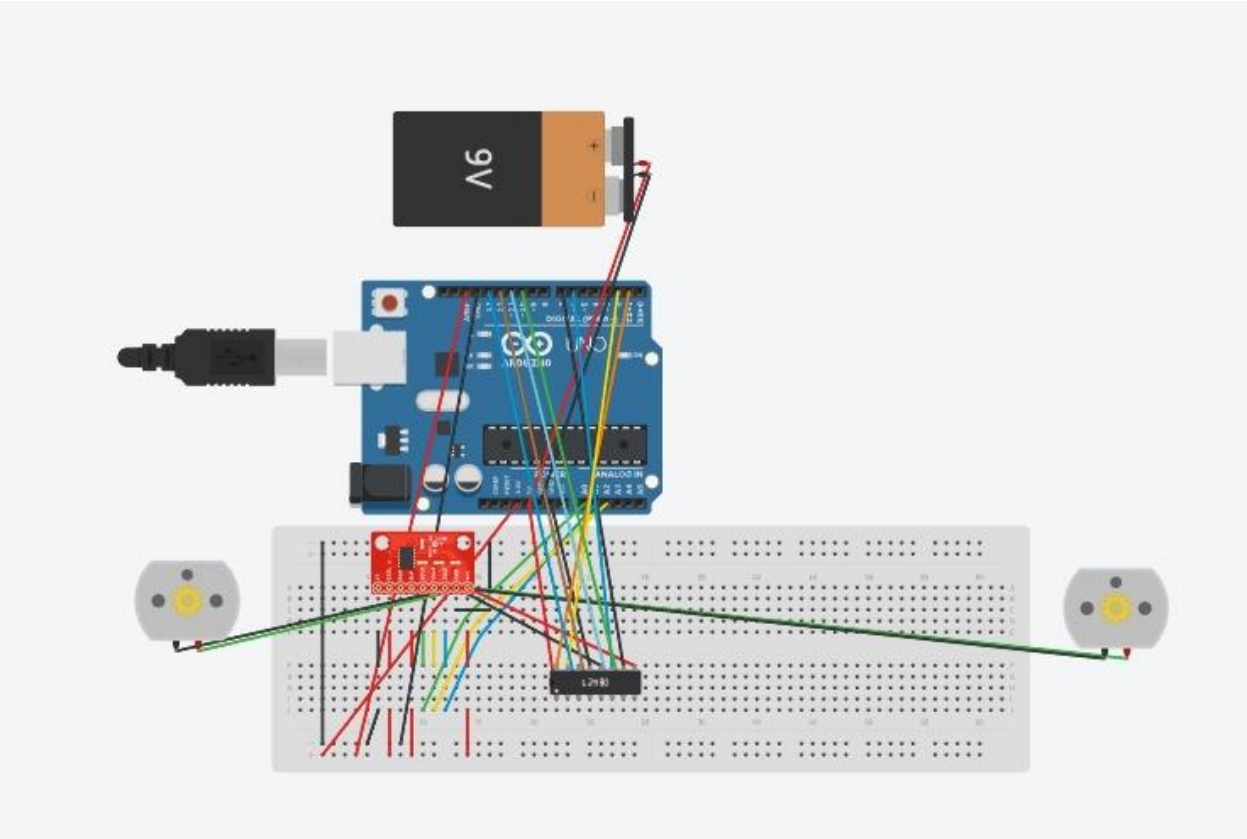

Fig:1 - Design of a Self-Balancing Robot

5. Code / Program

```
#include <Wire.h>

#include <MPU6050.h>

MPU6050 mpu;

float angle = 0;

float previousAngle = 0;

float Kp = 60;
```

```
float Ki = 70;
float Kd = 1.4;

float error = 0;
float previousError = 0;
float integral = 0;
float derivative = 0;
float output = 0;

void setup()
{
    Serial.begin(9600);
    Wire.begin();
    mpu.initialize();
}

void loop()
{
    int16_t ax, ay, az;
    int16_t gx, gy, gz;
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    float accelAngle = atan2(ay, az) * 180 / PI;
    float gyroRate = gx / 131.0;
    angle = 0.98 * (previousAngle + gyroRate * 0.01) + 0.02 * accelAngle;
    error = 0 - angle;
    integral += error;
```

```
    derivative = error - previousError;

    output = Kp * error + Ki * integral + Kd * derivative;

    previousError = error;

    previousAngle = angle;

    Serial.println(output);


    delay(10);

}
```

6. Output / Observations

After uploading the program to the microcontroller, the MPU6050 sensor continuously measures the robot's tilt angle using accelerometer and gyroscope data. These values are processed using a complementary filter to obtain the accurate angle of inclination.

The calculated angle is then compared with the desired reference angle (0°), which represents the perfectly upright position of the robot. The difference between the measured angle and the reference angle is considered as the error.

The PID controller processes this error and generates an output signal. This signal determines how the motors should rotate in order to correct the robot’s position and maintain balance.

The system behavior was observed through the Arduino Serial Monitor, where the current angle, error, and PID controller output were displayed continuously.

During the experiment, the following observations were made:

- When the robot was tilted forward or backward, the angle value changed, indicating the direction and magnitude of the tilt.
- The error value increased when the robot moved away from the upright position.
- The PID controller generated corrective output to drive the motors in the appropriate direction.
- As the robot approached the balanced position, the error gradually decreased.
- When the robot became stable, the PID output approached zero, indicating that only minimal correction was needed to maintain balance.

```
Self Balancing Robot System Started
Reading sensor data...

Angle: 3.25 deg | Error: -3.25 | PID Output: -195.40
Angle: 2.80 deg | Error: -2.80 | PID Output: -168.22
Angle: 2.10 deg | Error: -2.10 | PID Output: -124.30
Angle: 1.55 deg | Error: -1.55 | PID Output: -92.70
Angle: 0.90 deg | Error: -0.90 | PID Output: -54.80
Angle: 0.35 deg | Error: -0.35 | PID Output: -21.50

Robot approaching balanced position...

Angle: 0.10 deg | Error: -0.10 | PID Output: -6.20
Angle: 0.05 deg | Error: -0.05 | PID Output: -3.10
Angle: -0.02 deg | Error: 0.02 | PID Output: 1.20
Angle: 0.01 deg | Error: -0.01 | PID Output: -0.60

Robot is balanced and stable.

Angle: 0.00 deg | Error: 0.00 | PID Output: 0.05
Angle: -0.01 deg | Error: 0.01 | PID Output: 0.30
Angle: 0.02 deg | Error: -0.02 | PID Output: -0.45
```

Fig: 2 - Output of serial monitor

Observed behavior: -

Condition	Robot Response
Robot tilts forward	Wheels rotate forward
Robot tilts backward	Wheels rotate backward
Robot stands upright	Motors remain stable

Table:1 - Observed

Sensor outputs such as acceleration and angular velocity values were observed through the Arduino Serial Monitor.

6. Result

The self-balancing robot was successfully implemented using the ATMEGA328P microcontroller, MPU6050 sensor, and L298N motor driver. The robot was able to maintain its balance automatically using a PID control algorithm.

The system successfully detected the robot's orientation and adjusted the motor speed to keep the robot upright.

Therefore, the objective of the experiment was achieved.

8. Conclusion

In this experiment, we successfully designed and implemented a self-balancing robot using sensor feedback and control algorithms. The robot was able to detect its tilt using an accelerometer and gyroscope and maintain stability using a PID control mechanism.

Through this project, we learned:

- How motion sensors work
- Implementation of control systems in robotics
- The importance of PID controllers in balancing systems
- Practical integration of hardware and software in robotics

This project demonstrates an important concept used in many real-world applications such as Segway vehicles, delivery robots, and autonomous balancing systems.